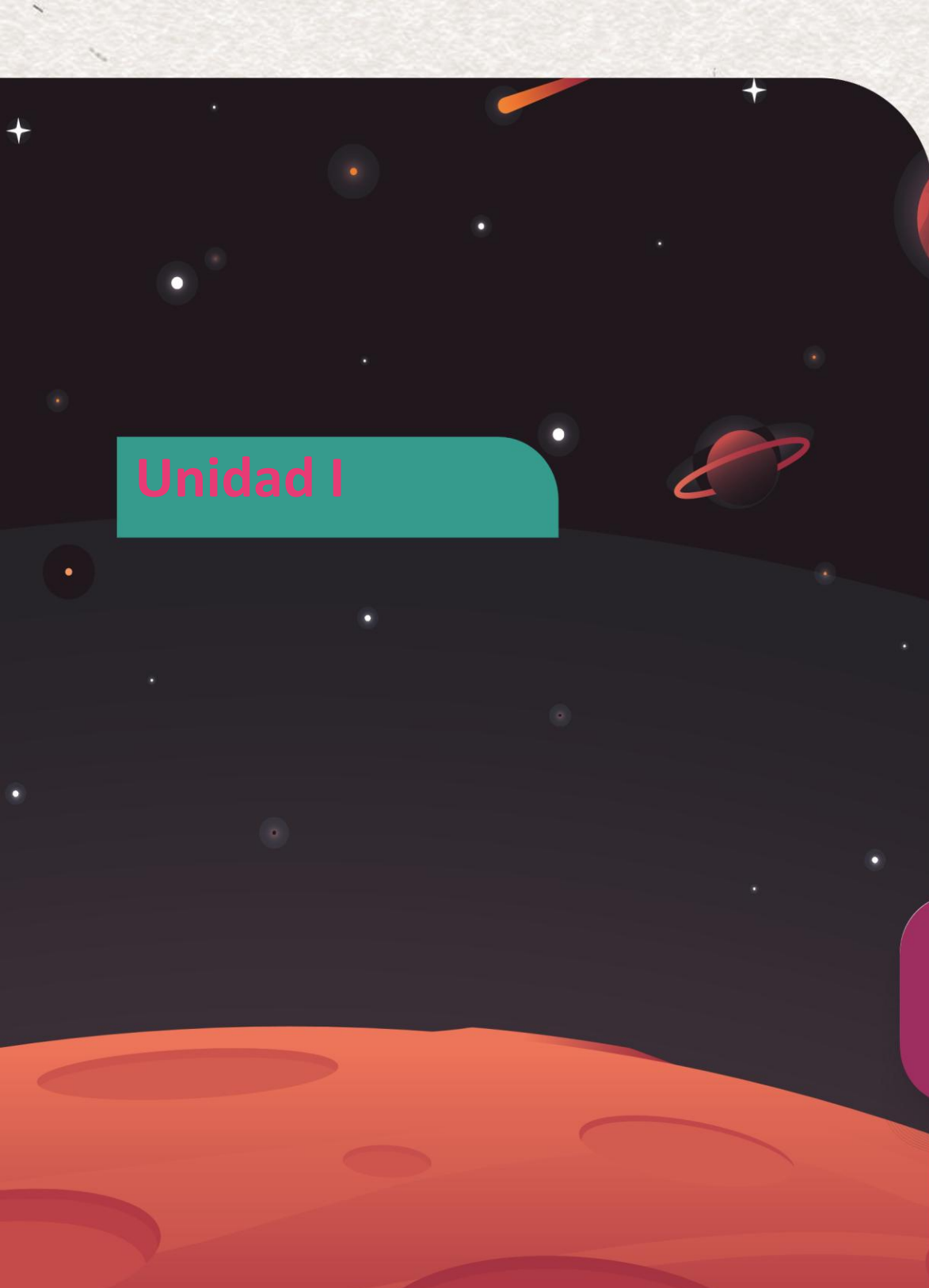




CESDE

Santiago Yosa González

Ingeniero Sistemas de Información



Unidad I

Funciones en JavaScript:

Funciones flecha

Funciones anónimas.

Estructuración del sitio Web

Situación

En la sesión anterior, aprendimos a crear una función así: `function calcularImpuesto() { ... }`. Esto es como escribir una receta en una página completa de tu libro y ponerle un título. Es formal y reutilizable.

Pero ¿qué pasa cuando tienes una tarea muy simple y de un solo uso? Por ejemplo, cuando programas un botón: `miBoton.addEventListener('click', ...)`

¿Realmente necesitas ir a tu libro, escribir una receta completa llamada `mostrarUnMensajeDeClick` y luego venir al botón y decirle que use esa receta?

Por ejemplo, imagina que tienes un robot. En lugar de guardar la instrucción "GirarIzquierda" en su memoria permanente, solo quieres decirle: "Robot, ejecuta *esta instrucción que te estoy pasando ahora mismo*".

Hoy aprenderemos que las funciones son **Valores**. Al igual que `let edad = 25;` (donde el valor es un Number) o `let saludo = "Hola";` (donde el valor es un String), podemos hacer esto:

```
let miInstruccion = /*...una función aquí...*/;
```


Funciones como Ciudadanos de Primera Clase

En JavaScript, las funciones no son "ciudadanos de segunda clase" (como en otros lenguajes, donde solo pueden ser definidas y llamadas). Aquí, son "de primera clase". Esto significa que una función es un **VALOR**.

Como cualquier otro valor (un Number, un String, un Boolean), una función puede:

1. **Ser asignada a una variable** (Este es nuestro enfoque de hoy).
2. **Ser pasada como argumento** a *otra* función (Lo veremos en los ejemplos).
3. **Ser retornada** por *otra* función (Un concepto más avanzado que veremos en el futuro).

Expresión de Función (Function Expression)

Cuando aplicamos la regla #1 (asignar una función a una variable), creamos una **Expresión de Función**.

Las **Declaraciones** (function saludar() {}) son "elevadas" (hoisting). El motor de JS las "mueve" al principio del archivo. Puedes llamar a saludar() *antes* de haberla escrito en el código.

Las **Expresiones** (const decirAdios = ...) **NO** son elevadas. La variable decirAdios sigue las reglas de const: no puedes usarla antes de que sea declarada y asignada. Esto es, generalmente, un comportamiento más predecible y seguro.

Lo que sabíamos (Declaración):

```
function sumar(a, b) {  
  return a + b;  
}
```

Lo nuevo (Expresión):

```
const sumar = function(a, b) {  
  // ^      ^  
  // |      |  
  // Variable  Función Anónima (el valor)  
  // (const)  (fíjate que NO tiene nombre  
  //          después de 'function')  
  return a + b;  
}; // <-- ¡Punto y coma! Porque es el final de una asignación de variable.
```

Uso: Una vez guardada en la variable, la variable *se convierte* en la función. La llamamos usando el nombre de la variable. `let resultado = sumar(5, 10);` // resultado es 15

Pasar Funciones como Argumentos Podemos pasar una función anónima directamente como argumento a otra función que espera recibir una "instrucción".

Ejemplo con `addEventListener` (decirle a un botón qué hacer al ser presionado):

```
// Obtenemos el botón
```

```
const miBoton = document.getElementById("miBoton");
```

```
// Le pasamos una función anónima como segundo argumento.
```

```
// Esta función se ejecutará SÓLO cuando ocurra el 'click'.
```

```
miBoton.addEventListener("click", function() {
```

```
  console.log("¡Me hicieron clic!");
```

```
});
```

Pausa para cuestionar: ¿Vemos por qué no necesitamos un nombre?

La función solo existe para ser entregada a `addEventListener`. No la volveremos a llamar desde ningún otro lugar. Darle un nombre sería innecesario.

Funciones Flecha: La versión compacta y moderna

Introducidas en ES6 (ECMAScript 2015), las **funciones flecha** son una sintaxis más corta y elegante para escribir funciones, especialmente las anónimas. Son la forma preferida de escribir funciones en el JavaScript moderno, ya que eliminan la necesidad de las palabras clave `function` y `return` en muchos casos.

Las **Funciones Flecha** (`=>`) son *siempre* anónimas y se usan *siempre* como expresiones.

Analogía: La Evolución de un Mensaje

- **Declaración:** `function saludar() { ... }` (Un email formal).
- **Expresión:** `const saludar = function() { ... }` (Un mensaje de texto completo).
- **Flecha:** `const saludar = () => { ... }`

```
const sumarFlecha = (a, b) => { return a + b; };
```

Sintaxis Paso a Paso:

Empezamos con una Expresión de Función tradicional:

```
const sumar = function(a, b) {  
  return a + b;  
};
```

Paso 1: Quitar la palabra function.

```
const sumar = (a, b) { // <- ¡Esto es un error de sintaxis!  
  return a + b;  
};
```

Paso 2: Añadir la "flecha" (=>) entre los parámetros y el cuerpo.

```
const sumar = (a, b) => {  
  return a + b;  
};
```


Anatomía del Código

```
const sumar = (a, b) => {  
  //      ^      ^  
  //      |      |  
  //  Parámetros  Flecha  
  //              (indica el inicio  
  //              del cuerpo)  
  return a + b;  
};
```

Las Reglas de la Sintaxis Corta (Optimizaciones):

Las funciones flecha tienen 3 reglas que las hacen aún más cortas.

Regla 1: Si tienes UN SOLO PARÁMETRO, los paréntesis () son opcionales.

```
// Larga  
const duplicar = (n) => {  
  return n * 2;  
};
```

```
// Corta (Regla 1)  
const duplicar = n => {  
  return n * 2;  
};
```

Regla 2: Si NO TIENES PARÁMETROS, los paréntesis () son OBLIGATORIOS.

```
const saludar = () => {  
  return "Hola";  
};
```

Regla 3: El Retorno Implícito (LA MÁS IMPORTANTE).

- Si el cuerpo de tu función (lo que está dentro de {}) contiene **UNA SOLA LÍNEA...**
- ...y esa línea es un return...
- ...puedes **eliminar** las llaves {} y la palabra return.

El valor de esa única línea se retorna automáticamente.

Ejemplo 1 (Reglas 1 y 3):

```
// Larga  
const duplicar = (n) => {  
  return n * 2;  
};
```

```
// Aplicando Regla 3 (Quitar {} y return)  
const duplicar = (n) => n * 2;
```

```
// Aplicando Regla 1 (Quitar ())  
const duplicar = n => n * 2;
```

Anatomía (Retorno Implícito):

```
const duplicar = n => n * 2;  
//           ^           ^  
//           |           |  
//      Parámetro Expresión que se retorna
```

Ejemplo 2 (con 2 parámetros):

```
// Larga  
const sumar = (a, b) => {  
  return a + b;  
};
```

```
// Aplicando Regla 3 (Retorno Implícito)  
const sumar = (a, b) => a + b;
```

Ejemplo 3 (sin parámetros):

```
// Larga  
const saludar = () => {  
  return "Hola";  
};
```

```
// Aplicando Regla 3 (Retorno Implícito)  
const saludar = () => "Hola";
```

Preguntas Frecuentes

¿Significa esto que *nunca* debo usar `function` y siempre usar `=>`? No. Debes usar la herramienta correcta.

- **Usa `=>` (Flecha):** En el 90% de los casos, especialmente cuando pasas una función como argumento a otra (en `addEventListener`, `.map()`, `.filter()`, etc.) y cuando necesitas heredar el `this` de la clase o el objeto contenedor.
- **Usa `function` (Regular):** Cuando *necesitas* que la función tenga su propio `this` dinámico. El caso más común son los **métodos principales** de un objeto literal (como `procesarItems` en el ejemplo anterior) o en constructores antiguos.

¿Por qué las Expresiones de Función (`const f = ...`) son más seguras que las Declaraciones (`function f...`)?

Por el *Hoisting*. En un archivo grande, una Declaración de Función puede ser llamada desde cualquier lugar, incluso antes de que la leas, lo cual puede ser confuso. Una Expresión de Función (`const f = ...`) no se puede usar hasta que el código la haya definido. Esto fuerza un orden lógico y previene errores donde se usa una función antes de que esté "lista" (o antes de que el programador mentalmente espere que esté lista).

Ejemplo 1: Funciones Anónimas Tradicionales (Expresiones)

```
// Guardamos una función anónima en la variable 'multiplicar'  
const multiplicar = function(a, b) {  
  return a * b;  
};
```

```
// Guardamos otra en 'dividir'  
const dividir = function(a, b) {  
  if (b === 0) {  
    return "Error: no se puede dividir por cero";  
  }  
  return a / b;  
};
```

```
// Las usamos  
console.log( "Multiplicación (5x7):", multiplicar(5, 7) );  
console.log( "División (10/2):", dividir(10, 2) );  
console.log( "División (10/0):", dividir(10, 0) );
```

Ejemplo 2: Refactorizando a Funciones Flecha (Sintaxis Moderna) Ahora, reescribamos las mismas funciones usando la sintaxis de flecha.

```
const sumar = (a, b) => a + b;  
const restar = (a, b) => a - b;  
const multiplicar = (a, b) => a * b;  
  
// 'dividir' necesita un 'if', así que usamos sintaxis larga.  
const dividir = (a, b) => {  
  if (b === 0) {  
    console.error("¡No se puede dividir por cero!");  
    return null; // O algún valor de error  
  }  
  return a / b;  
};  
  
// Probamos una:  
console.log( sumar(10, 5) ); // Output: 15
```

Ejemplo 3: Pasando una función como argumento Este es el concepto más avanzado, pero podemos hacerlo sin elementos externos. Crearemos una función que *recibe* otra función como instrucción. Siguiendo el ejemplo anterior:

```
// Esta función es "genérica". No sabe qué operación hará.  
// Acepta dos números y una "operacion" (que esperamos sea una función).  
  
function ejecutarCalculo(num1, num2, funcionOperacion) {  
  
    // 1. Recibe los números y la FUNCIÓN.  
    console.log("Ejecutando un cálculo...");  
  
    // 2. ¡Llama a la función que recibió como argumento!  
    // y le pasa los números.  
    const resultado = funcionOperacion(num1, num2);  
  
    // 3. Retorna el resultado.  
    return resultado;  
}
```


Conectamos todo. Ahora llamamos a ejecutarCalculo y le *pasamos* nuestras funciones-herramienta.

```
// Le pasamos la *variable* que *contiene* la función 'sumar'  
let resultadoSuma = ejecutarCalculo(20, 10, sumar);  
console.log(resultadoSuma); // Output: 30
```

```
// Le pasamos la *variable* 'restar'  
let resultadoResta = ejecutarCalculo(20, 10, restar);  
console.log(resultadoResta); // Output: 10
```

```
// Le pasamos la *variable* 'dividir'  
let resultadoDiv = ejecutarCalculo(20, 0, dividir);  
// Output: ¡No se puede dividir por cero!  
// Output: null
```

Pasando la Función Anónima Directamente (El "Post-it"):

¿Qué pasa si no quiero crear una variable multiplicar solo para usarla una vez? Puedo pasar la función *directamente* en el argumento.

Versión 1: Anónima Tradicion

```
let resultadoMult = ejecutarCalculo(7, 5, function(a, b) {  
  return a * b;  
});  
console.log(resultadoMult); // Output: 35
```

Versión 2: Función Flecha (La forma moderna y preferida)

```
let resultadoPotencia = ejecutarCalculo(2, 5, (a, b) => a ** b);  
console.log(resultadoPotencia); // Output: 32
```

Práctica Acompañada

Práctica 1: Refactorizando: Convertir funciones tradicionales en funciones flecha modernas.

Paso 1: Observa la función inicial. Es una expresión de función que formatea un saludo.

```
const saludar = function(nombre) {  
  return "¡Hola, " + nombre + "!";  
};
```

```
// Pruébala  
console.log( saludar("Estudiante") );
```

Analiza la función.

- ¿Cuántos parámetros tiene? = Uno, nombre.
- ¿Cuántas líneas de código tiene en su cuerpo? = Una.
- ¿Esa línea es un return? = Respuesta: Sí.

Conclusión: Es un candidato perfecto para una función flecha con retorno implícito y sin paréntesis en el parámetro.

Escribe la versión flecha.

Empieza con `const saludarFlecha =`.

Añade el parámetro (sin paréntesis): `nombre`.

Añade la flecha: `=>`.

Añade la expresión que debe retornar (sin `return` y sin `{}`): `"¡Hola, " + nombre + "!"`.

Trata de cambiarla y usar template literals

// La nueva versión Flecha

```
const saludarFlecha = nombre => `¡Hola, ${nombre}!`;
```

// Comprobación

```
console.log( saludarFlecha("Estudiante Flecha") );
```

Práctica 2: El Validador

Crear una función flecha desde cero y usarla.

Paso 1: Define la tarea. Queremos crear una función llamada esPasswordValido.

- Debe tomar un parámetro: password (un String).
- Una contraseña es válida si su longitud (propiedad .length que vimos en Semana 1) es **mayor que 8 caracteres**.
- Debe retornar un booleano (true o false).
- ¡Debes escribirla usando la sintaxis de **función flecha y retorno implícito**!

Paso 2: Escribe la función.

- Empieza con `const esPasswordValido =`.
- Añade el parámetro (¿necesita paréntesis?).
- Añade la flecha `=>`.
- Escribe la *expresión de comparación* que retorna true o false.

Paso 3: Prueba tu función.

- Prueba con "1234" (debería dar false).
- Prueba con "passwordseguro123" (debería dar true).

```
const esPasswordValido = password => password.length > 8;
```

Ejercicio Independiente

El Sistema de Salarios

Contexto: Estás programando un módulo simple de recursos humanos. Necesitas calcular los impuestos y el salario neto.

Instrucciones Claras:

Tarea 1: El Calculador de Impuestos

- Crea una **expresión de función** llamada calcularImpuesto.
- Debe usar la sintaxis de **Función Flecha** (=>).
- Debe tomar un parámetro: salarioBruto.
- Debe calcular un impuesto fijo del 10% sobre el salario.
- Debe usar un **retorno implícito**.

Tarea 2: El Calculador de Salario Neto

- Crea una **expresión de función** llamada calcularSalarioNeto.
- Debe usar la sintaxis de **Función Flecha** (=>).
- Debe tomar un parámetro: salarioBruto.
- Esta función **debe llamar** a tu función calcularImpuesto para obtener el monto del impuesto.
- Debe retornar el salarioBruto menos el impuesto calculado.
- *(Pista: Como esta función necesita dos pasos –llamar a la primera y luego restar–, no puede usar un retorno implícito. Deberás usar {} y return).*

Tarea 3: Prueba

- Declara una variable salario con un valor (ej. 5000000).
- Llama a calcularSalarioNeto pasándole ese salario.
- Imprime el resultado en consola.

Criterios de Logro Esperados:

- Se crea calcularImpuesto como una función flecha de una sola línea (retorno implícito).
- Se crea calcularSalarioNeto como una función flecha de múltiples líneas (con {} y return).
- calcularSalarioNeto reutiliza (llama) correctamente a calcularImpuesto.
- Si se prueba con 5000000, la consola debe imprimir 4500000.

CESDE | comfama